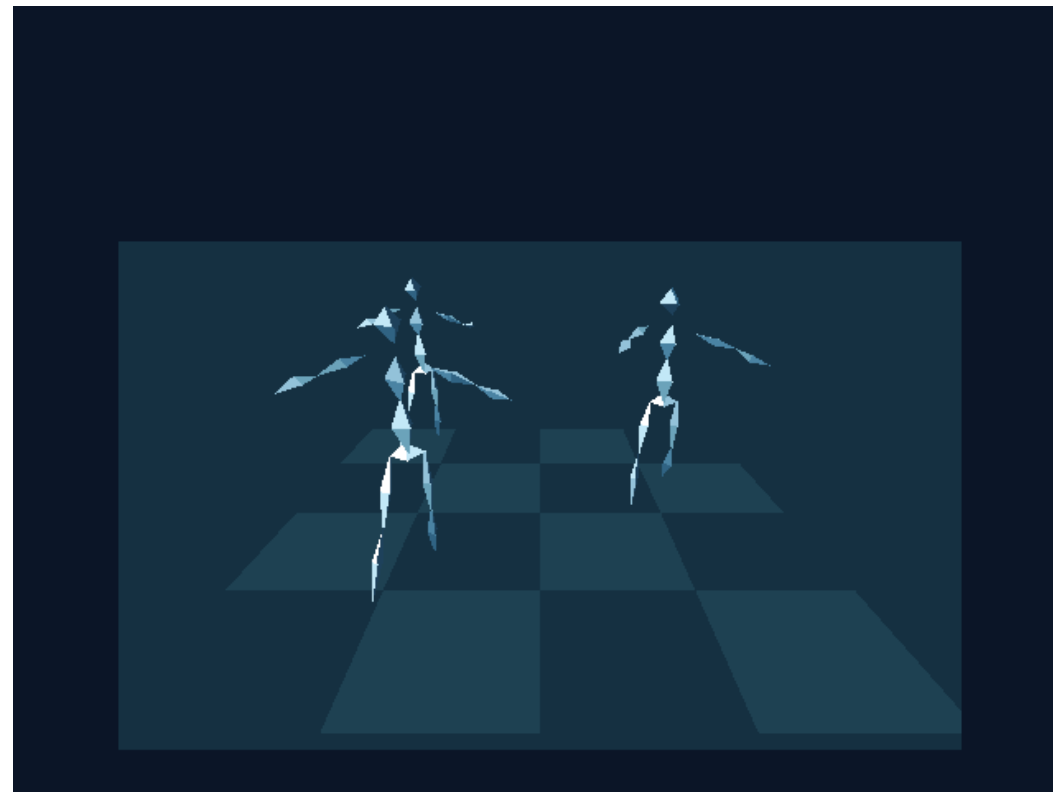


BVHの動きをMSXで描く

関節の階層から、V9968 + geo3dの立体ダンスへ

この資料で学ぶこと

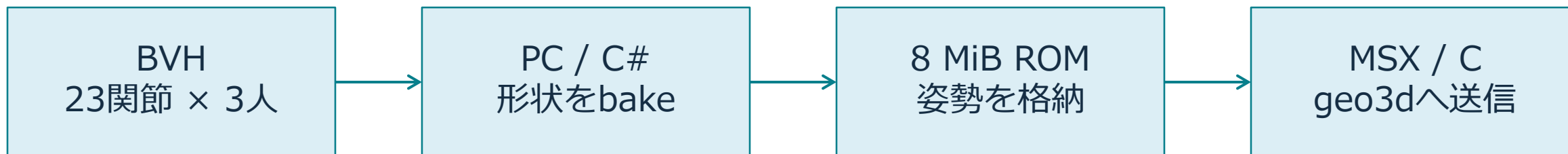
1. BVHに入っている情報
2. bakeで計算する内容
3. 再生時のgeo3dの役割



SCREEN 7 FIL・512 × 424・16色

bakeは繰り返し計算をPCへ移す

一度だけ形状を計算し、再生時には姿勢を選んで描きます。



PC : テキスト解析、関節の変換合成、メッシュ作成、法線計算。

MSX : 時刻の選択、ROMバンク読み出し、カメラ設定、形状送信。

BVHは骨格と動作の数値を保存する

画像や完成済みのキャラクターモデルは入っていません。

HIERARCHY

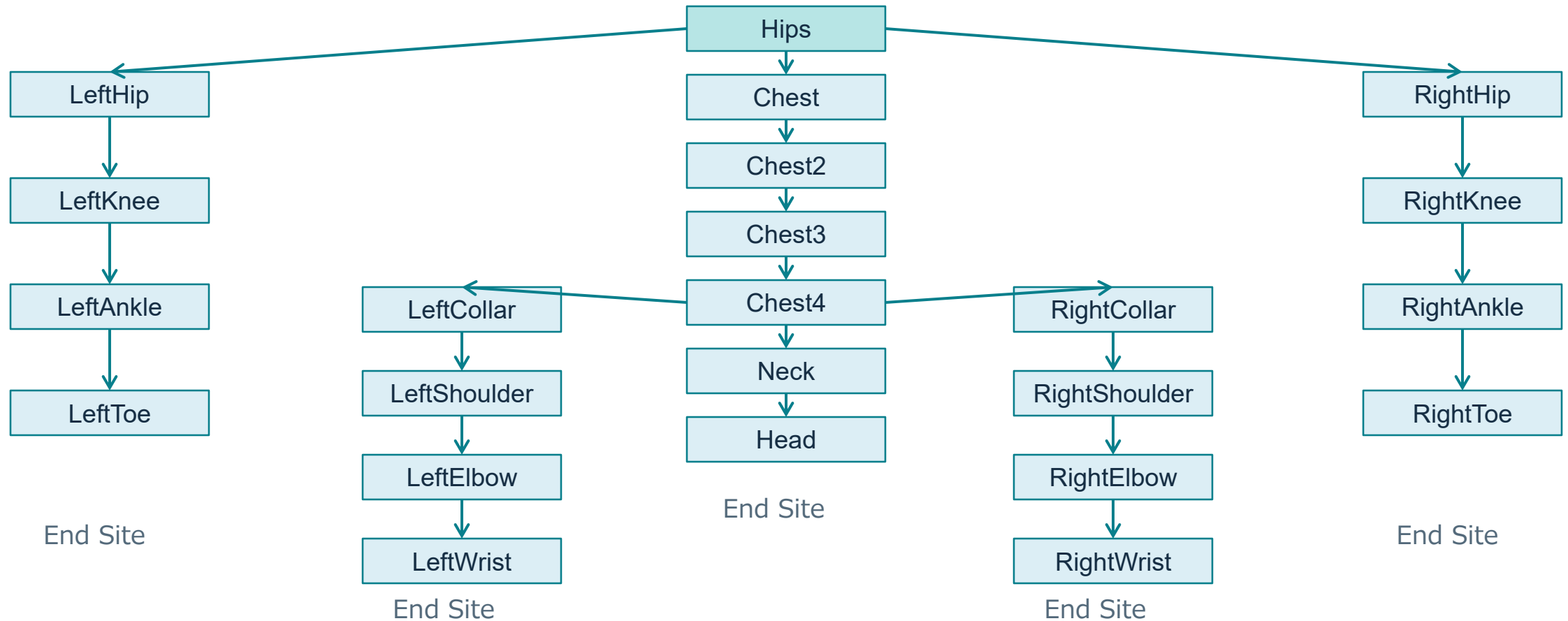
ROOT / JOINT : 親子関係のある関節
OFFSET : 親からの距離
CHANNELS : 各フレームの数値の並び順
End Site : 動作チャンネルを持たない末端

MOTION

2820フレーム × 0.025秒 = 70.5秒
各行に位置・回転のチャンネル値が並びます。
3人分とも想定した構造かどうか検証します。

実データの23関節の階層

矢印は「親 → 子」。End Siteは関節数に含めない末端です。



回転は宣言された順番で読む

このBVHの宣言は Yrotation → Xrotation → Zrotation です。

```
for (int k = 0; k < j.channels.Count; k++)  
    if (j.types[k].EndsWith("rotation"))  
        local = Mul(local, Rotate(j.types[k],  
            clip.motion[frame][j.channels[k]]));
```

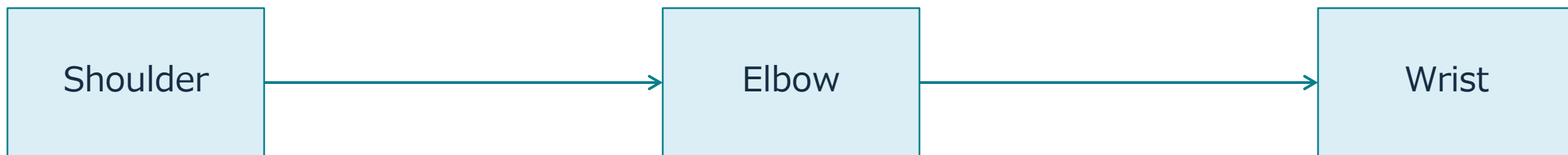
OFFSETは固定値。
チャンネル値は毎フレーム
変化。

Y軸→X軸の回転と、
X軸→Y軸の回転では、
通常、結果が異なります。

L = 平行移動 × Ry × Rx × Rz

子の姿勢は親の変換を引き継ぐ

順運動学：根元から末端へ向かって計算します。



```
world[i] = j.parent < 0 ? local  
          : Mul(world[j.parent], local);
```

$$W_{\text{子}} = W_{\text{親}} \times L_{\text{子}}$$

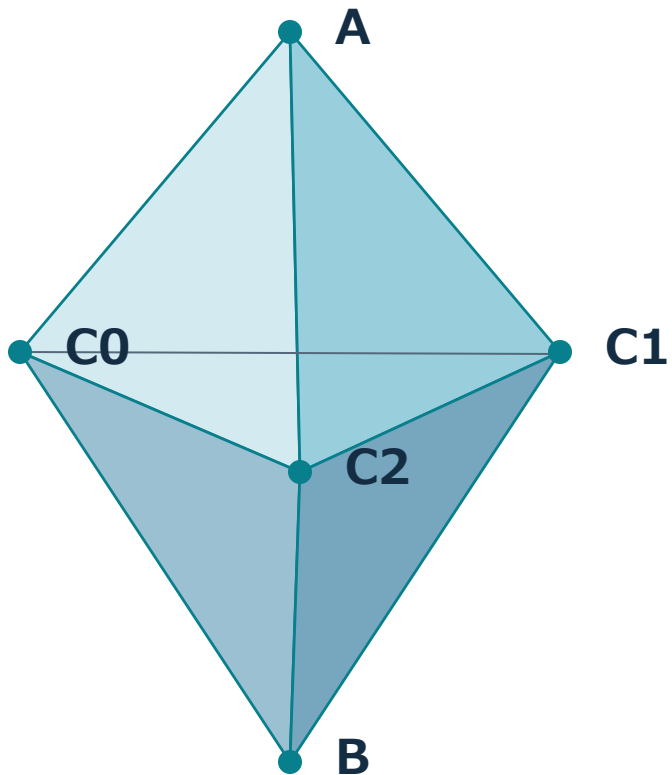
肩を回すと、肘も手首も移動します。

肘を回すと、その先の手首が追従します。

Point(world[j], position)で共通の空間内の位置を求めます。

選んだ骨を小さな立体にする

三角両錐：5頂点・6枚の三角形で構成します。



1人あたり12部品
= 60頂点 / 72面

3人分
= 180頂点 / 216面

各部品は単純な立体です。
滑らかなスキニングは行いません。

再生前に外向きの法線を計算する

法線は、光に対して面がどちらを向くかをgeo3dへ伝えます。

```
V n = (p.vertices[ib] - p.vertices[ia])  
      .Cross(p.vertices[ic] - p.vertices[ia]);  
p.faces.Add(new Face {  
    a = ia, b = ib, c = ic, n = n.Unit()  
});
```

引用を省いた向き判定で、内向きの法線と頂点順を反転します。

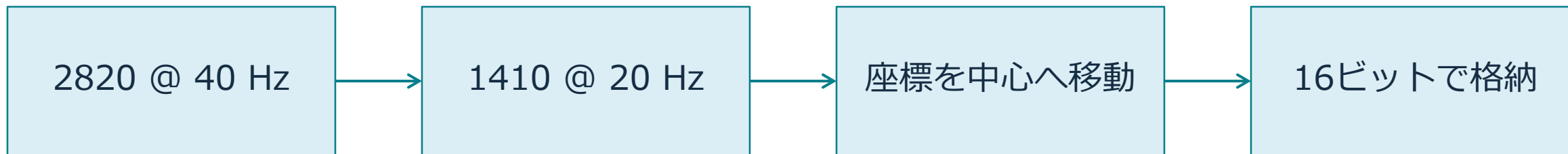
外積 → 面に垂直な方向
正規化 → 長さを1にそろえる

法線の保存形式
符号付き Q2.14
1.0は16384として保存。

座標：符号付き16ビット整数

40 Hzの入力を20 Hzの姿勢に変換

サンプリング・中心移動・整数化はPC側で行います。



```
Body(p, clips[actor], f * 2);  
V center = (lo + hi) * .5;  
V r = q - center;  
w.Write(Quant(r.x));  
w.Write(Quant(r.y));  
w.Write(Quant(r.z));
```

元の2フレームごとに1姿勢を採用。
全体の長さは70.5秒のままです。

全区間の範囲から中心を決定。
毎フレーム座標の原点を
置き直すことはありません。
姿勢補間の実装していません。

1姿勢は3456バイト

geo3dの頂点・面ストリームへ送れる順番で保存します。

頂点

$$180 \times 6 = 1080 \text{ bytes}$$

面

$$216 \times 11 = 2376 \text{ bytes}$$

頂点 / 6バイト

X・Y・Z：符号付きint16
下位バイトから保存します。
。

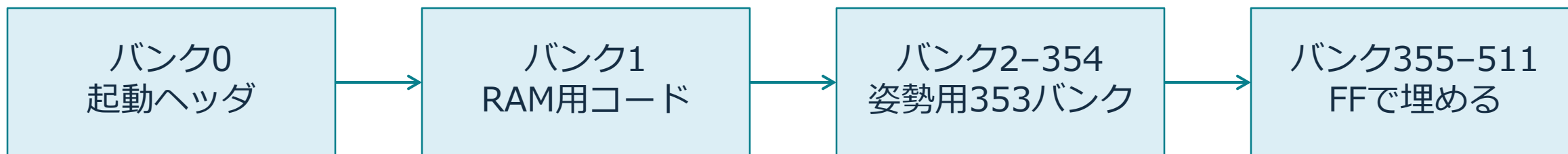
面 / 11バイト

I0・I1・I2・I3：uint8の頂点番号
NX・NY・NZ：Q2.14のint16法線
BASE：パレット番号1バイト

三角形(a, b, c)は、四角形レコード(a, b, c, c)で送ります。

16 KiBのROMバンクへ4姿勢を格納

ASCII16-Xで256バンクを超える領域を選択します。



姿勢のバンク

$4 \times 3456 = 13824$ バイト
+ 2560バイトの空き

最後の姿勢バンクは2姿勢のみ

```
unsigned int bank =  
    MOTION_BANK + frame / 4;  
*(volatile unsigned char *)  
    (0x6000 | (bank & 0x0f00)) =  
    (unsigned char)bank;  
pose = (const unsigned char *)  
    (0x4000 + (frame & 3) * FRAME_BYTES);
```

RAMで実行し、ROMから姿勢を読む

データバンクを切り替えても、実行コードは入れ替えません。

0000–3FFF	BIOS
4000–7FFF	ROM姿勢読み出し窓
8400–A06B	Cコード・変数・作業領域
E000–E2xx	IM2表・割り込み・起動補助
F300 ↓	スタック（下へ伸びる）

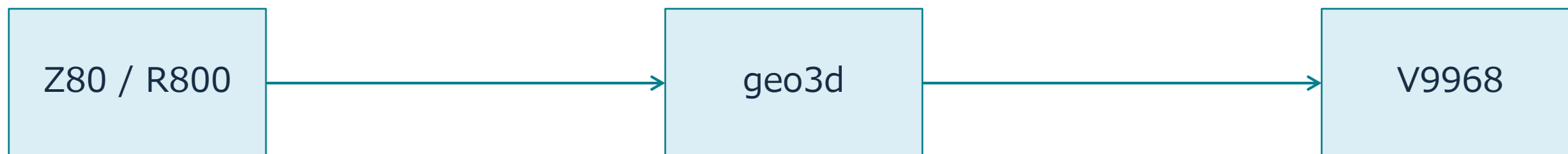
メインRAM : 64 KiB以上
VRAM : 256 KiB

割り込み処理もRAMに置き、姿勢のバンクを変更しません。

選択した姿勢を1つずつ読み出します。

CPU・geo3d・V9968の役割

geo3dが描画コマンドを作り、V9968が画素を書き込みます。



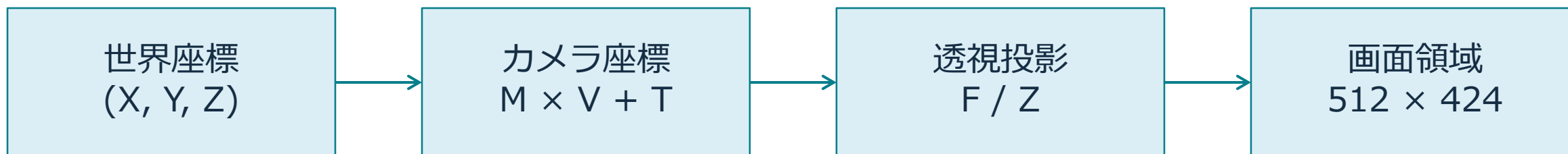
時間・入力・PSG
ROMバンク選択
カメラ設定と送信
画面クリア・表示切替

座標変換・透視投影
面の除去・明るさ計算
奥行き順に並べる
LINEコマンドを作る

コマンドエンジン
VRAMへ画素を描く
パレット・映像出力
SCREEN 7 FIL表示

透視投影で3D座標を画面へ写す

メッシュと別に、カメラ行列・平行移動を送ります。



```
geo_index = 0x18;  
word(SCREEN_FOCAL);           // 320  
word(SCREEN_WIDTH / 2);       // 256  
word(SCREEN_HEIGHT / 2);      // 212  
word(48);                      // near plane
```

$$\begin{aligned} SX &= CX + X \times F / Z \\ SY &= CY - Y \times F / Z \end{aligned}$$

世界座標のYは上向き。
画面座標のYは下向き。
遠くのもののほど小さく見えます。

ストリームを送ってRUNを実行

このアプリのgeo3dポートはインデックス9Dh・データ9Fhです。

```
reg(0x40, 0);  
geo_index = 0x50;  
stream(v, (unsigned int)nv * 6);  
reg(0x58, 0);  
geo_index = 0x52;  
stream(f, (unsigned int)nf * 11);
```

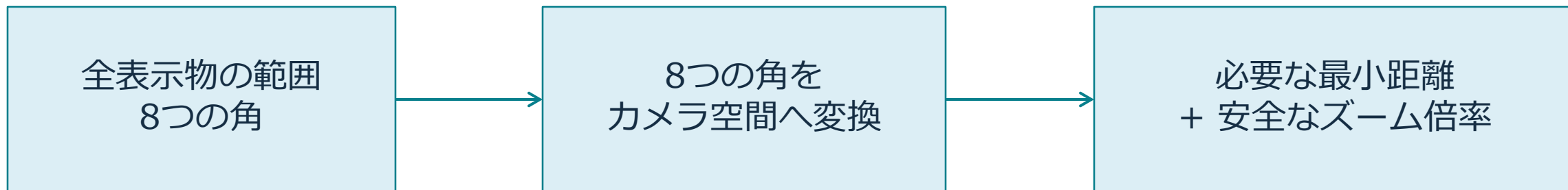
```
reg(0x42, nv); reg(0x59, nf);  
reg(0x48, 3);  
wait_geo();
```

40h / 58h : 書き込み位置
50h : 頂点ストリーム
52h : 面ストリーム
42h / 59h : 個数
48h = 3 : RUN + 面の塗り描画

RUN中は設定やVDP描画レジスタを変更しません。

見える姿勢全体を画面内へ収める

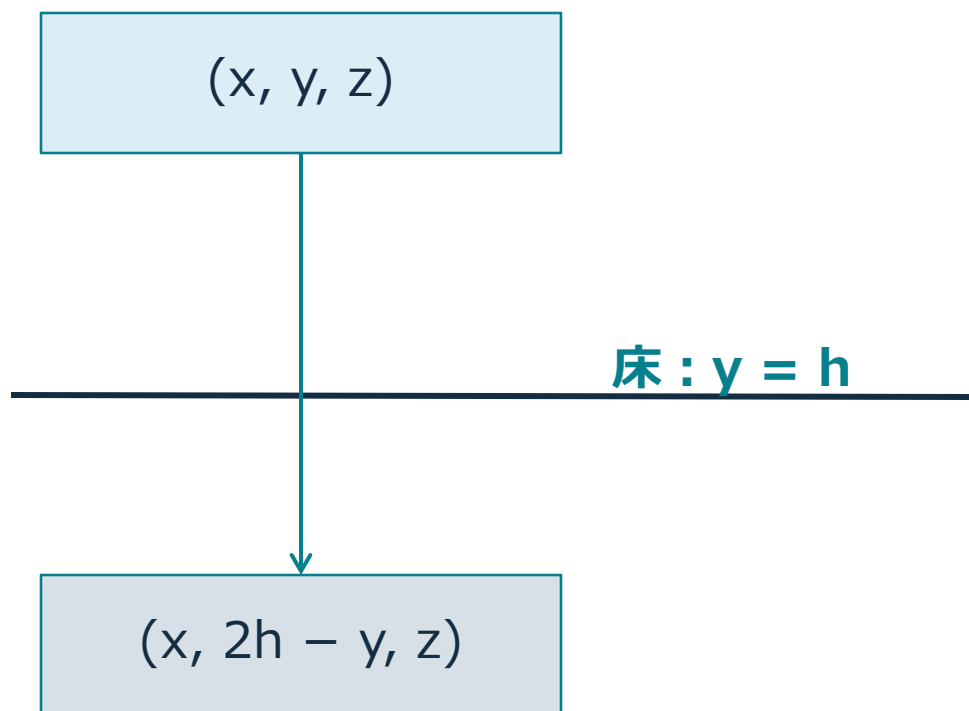
本体・残像・映り込みを合わせた範囲にカメラを合わせます。



投影後のX・Yの範囲と、手前の面からの距離を確認。
最も寄れるのは100%です。200%にするとカメラが遠ざかります。
水平回転は $\pm 45^\circ$ に制限します。

映り込みは形状を反転して描く

位置・頂点の回る順番・法線をセットで反転します。



$$h = \text{FLOOR_Y} = -174$$

法線 : $(nx, ny, nz) \rightarrow (nx, -ny, nz)$

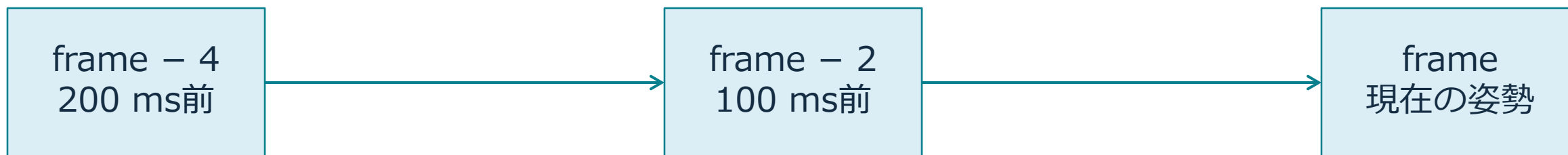
頂点順 : $(a, b, c, c) \rightarrow (a, c, b, b)$

暗いパレットの範囲を使います。

形状をもう一度描く方式です。物理的な反射や半透明合成ではありません。

過去の姿勢を軽く描いて残像にする

塗り面のコピーではなく、骨の両端を線で結びます。



各部品最初の2頂点が骨の両端です。
1人12本、3人で36本の線を描きます。
2段階の色で薄れる印象を作ります。

連続的なモーションブラーではなく、過去姿勢の残像表現です。

再生時間と描画速度を切り離す

60 HzのVBlank時計から、20 Hzの姿勢を選びます。

```
now = clock_ticks();
elapsed = now - previous;
previous = now;
playback_ticks += elapsed;
motion_frame = playback_ticks / 3;
```

1姿勢 = VBlankの3tick

1410姿勢 = 4230tick = 70.5秒

描画が遅いと姿勢を飛ばします。
ダンスの時間を意図的に延ばす方式
ではありません。

最後の0.5秒で
画面と音をフェード。

引用では一時停止・終端・範囲制限の判定を省略しています。

SCREEN 7 FILには2つの設定経路

CPUの違いではなく、V9968の実装に合うROMを選びます。

MOTION.rom

従来のopenMSXのFIL配置

R20 = 11h / R21 = 7Ah

FIL : R21 bit 6

表示時にY512 → Y0へコピー

R800 / Z80で

エミュレータ再生を検証済み。

MOTION-native.rom

現行FPGAのFIL配置

R20 = 31h / R21 = 3Ah

FIL : R20 bit 5

表示ページY0 / Y512を切替

ビルド確認済み。実機動作は未検証。

次に読むコードと変更の入口

一度に1つの役割から読み進めると理解しやすくなります。

tools/BvhBake.cs

BVH解析 → 関節変換 → 形状 → ROM

src/scene.c

バンク読み出し・カメラ・geo3d・表示効果

src/main.c

入力・再生時計・一時停止・再開始

src/video.c

SCREEN 7 FIL・パレット・ページ・白フェード

src/music.c

オリジナルのPSGループBGM

出典・クレジット・説明の範囲

非公式の独立したモーションビューア。ソース確認：2026-10-03。

BVH: Perfume global site project #001

<https://perfume-global.com/web/2012/03/perfume-global-site-project-001/>

geo3d: Alex Moncks / V9968_Cartridge

https://github.com/alexmoncks/V9968_Cartridge/tree/main/geo3d

メッシュとPSG音楽は独自作成。公式MP3は使用していません。
独自コードはライセンス未表明。GPL-2.0は2つの機種XMLのみに適用。
性能や実機互換性は、別途の測定・検証が必要です。